

Relatório 25 - Processamento de Linguagem Natural (NLP) (III)

Felipe Fonseca

1. Introdução

Nesses cards foram abordadas duas frentes do Processamento de Linguagem Natural (NLP), sendo a utilização do Tensor Flow para construir modelos capazes de aprender com sequências de texto, e a utilização do spaCy para analisar e extrair informações linguísticas de textos.

2. Desenvolvimento

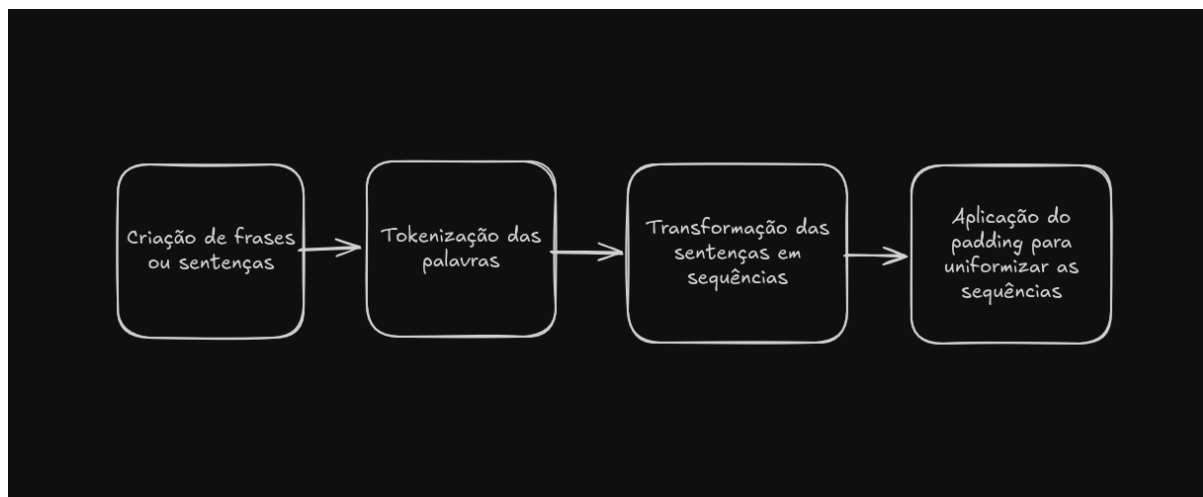
2.1 Tensor Flow

A primeira parte do card foi utilizando o tensor flow, e o início de tudo foi o processo de tokenização, onde em vez de transformar cada letra individualmente em um valor, transforma-se a palavra inteira em um número com significado, evitando confusão entre palavras diferentes que têm as mesmas letras, tendo como resultado um dicionário que mapeia cada palavra para um índice.

Após a tokenização, as sequências de índices resultantes possuem tamanhos diferentes, o que impede a formação de uma matriz uniforme para o treinamento. Para resolver isso, aplica-se o padding, que preenche as sequências com valores padrão (normalmente zero) até que todas tenham o mesmo comprimento.

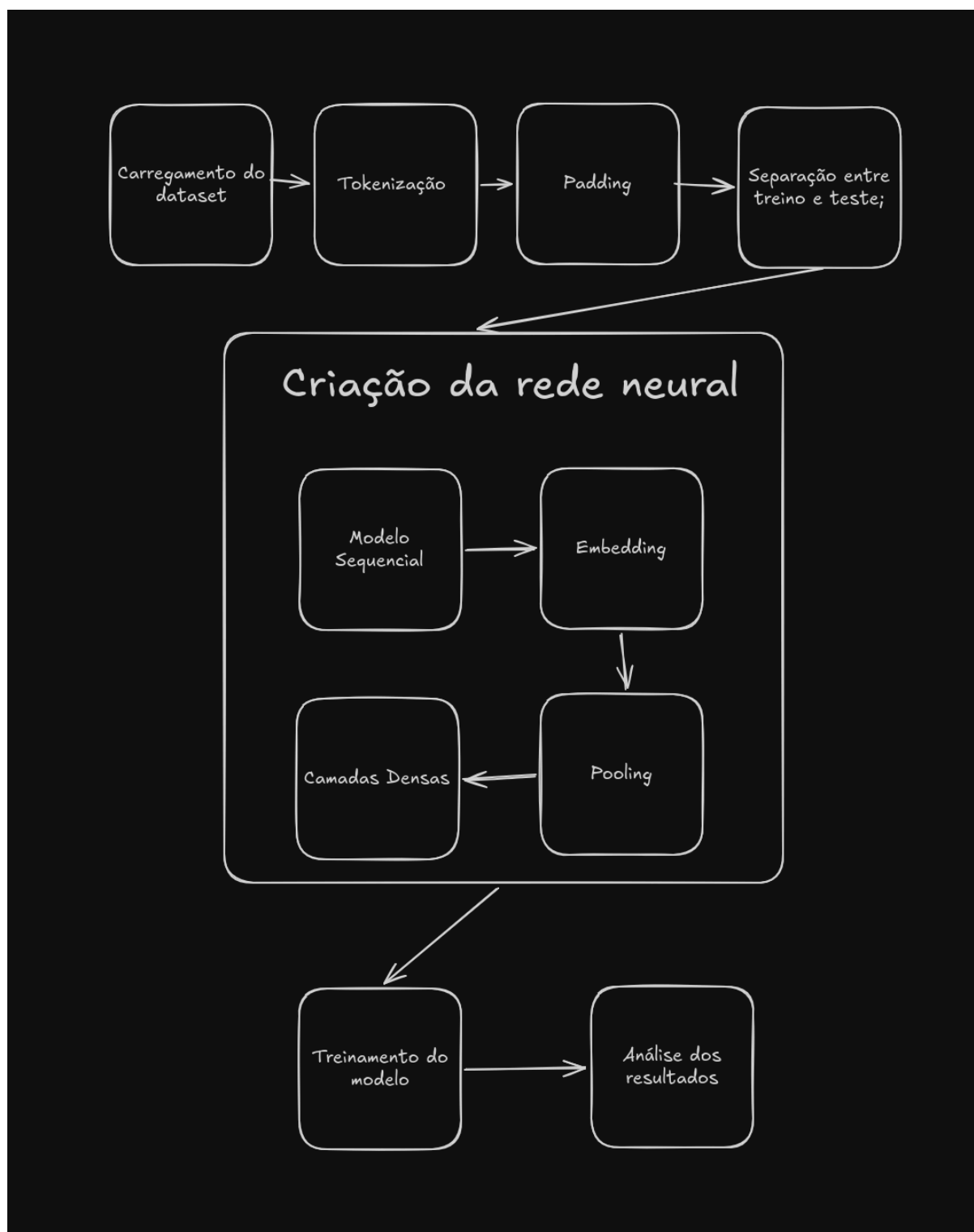
As principais variáveis que utilizamos para esse processo no tensor flow:

- `vocab_size`: tamanho máximo do dicionário de palavras;
- `embedding_dim`: quantidade de dimensões do embedding, ou seja, quantos números serão usados para representar cada palavra;
- `trunc_type`: define se o corte de sequências longas ocorre no início ou no final;
- `padding_type`: define se o preenchimento ocorre no início ou no final da sequência;
- `oov_tok`: valor atribuído a palavras não reconhecidas no vocabulário.



A imagem acima retrata o fluxo do código 1 feito no curso do tensor flow.

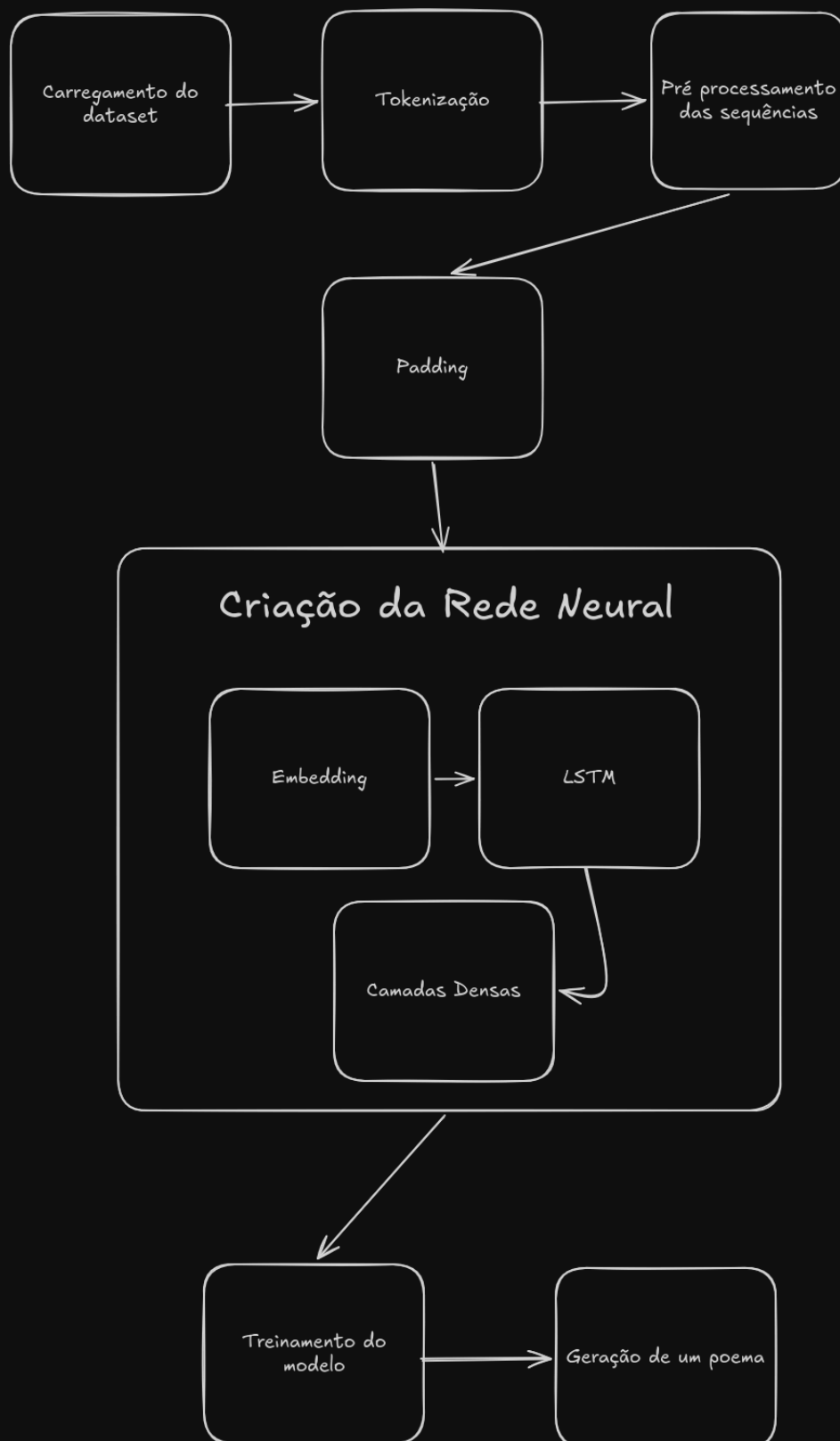
A segunda parte utilizou um dataset real para treinar um modelo de classificação binária (sarcasmo ou não):



Nessa parte foi apresentado o embedding, que em vez de representar cada palavra com um único número, o embedding atribui um vetor com múltiplas dimensões a cada palavra. Isso permite que o modelo capture contextos semânticos que um único número nunca conseguiria representar. Aqui a ordem das palavras não importa, pois trata-se de uma tarefa de classificação de sentimento, o que significa que é uma rede mais simples, sem memória entre passos.

A terceira parte do curso foi sobre a geração de texto, onde diferente da classificação de sarcasmo, aqui a sequência das palavras é importante, pois o modelo precisa prever a próxima palavra com base nas anteriores.

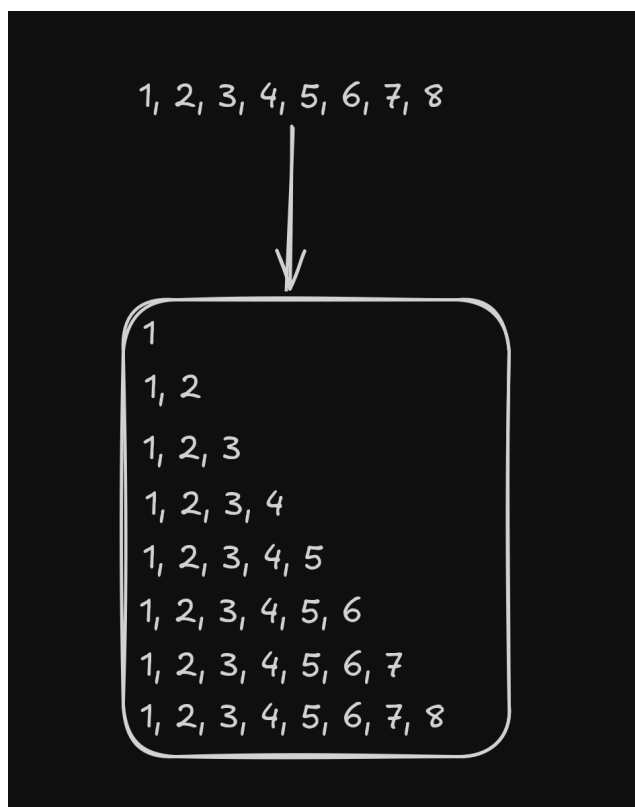
Para isso, foi utilizada uma Rede Neural Recorrente (RNN), que funciona diferente da rede tradicional, pois utiliza o resultado do passo anterior para calcular o próximo elemento, tornando-a naturalmente adequada para sequências. No entanto, redes RNN tendem a esquecer informações de passos muito anteriores. Por isso, foi introduzido a arquitetura LSTM (Long Short-Term Memory), que adiciona um cell state, que é uma espécie de memória de longo prazo que flui ao longo da rede, preservando informações relevantes por mais tempo. As camadas LSTM também podem ser utilizadas de forma bidirecional, permitindo que o modelo analise a sequência tanto da esquerda para a direita quanto ao contrário, capturando informações mais precisas.



O dataset utilizado foi um que contém letras de poemas;

Aplicamos a tokenização, mapeando as palavras em números;

Pré Processamento das Sequências: essa foi uma das partes mais interessantes, que foi ver como realmente é feito o treinamento da rede neural. A técnica utilizada é quebrar a sequência, em várias sequências menores, conforme na imagem abaixo:



Isso é importante, pois o modelo irá utilizar a próxima palavra para prever a anterior;

Aplicação do padding;

Criação da rede neural com camadas de embedding, LSTM e camada densa de saída;

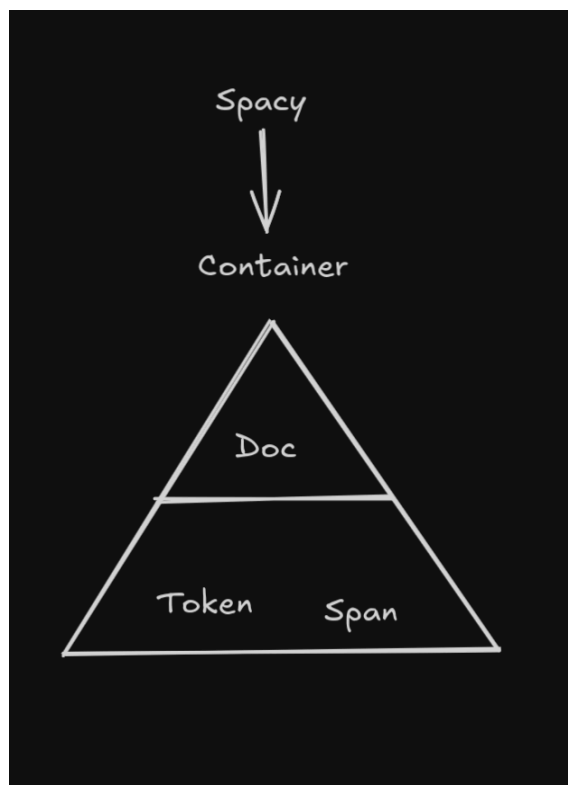
Treinamento e análise do histórico;

Geração de um poema com 100 palavras a partir de uma frase inicial fornecida pelo usuário.

2.2 spaCy

O spaCy é um framework de processamento de linguagem natural que tem como objetivo fazer com que sistemas compreendam a linguagem humana. Sua estrutura

central é um Container, que guarda todos os dados sobre um texto, sendo os três principais objetos o Doc, o Token e o Span.



O Doc é o principal objeto de processamento, pois ele engloba todo o documento e, ao processar o texto, realiza uma tokenização avançada que separa sinais, hifens, acentos e outros caracteres especialmente. Além disso, o Doc realiza o reconhecimento de entidades nomeadas (NER), identificando e categorizando elementos como pessoas, localizações, organizações, datas etc. Um Doc é composto por Tokens, que podem ser palavras, sinais, números e até pontuações, e cada um com atributos linguísticos como Part of Speech (classe gramatical), dependência sintática, head (a palavra à qual o token está subordinado), morph (propriedades como tempo verbal e forma) e lemma (a forma base da palavra no dicionário). Já um Span é uma fatia de um Doc, podendo representar uma frase ou uma entidade, e pode ser categorizado de forma mais específica conforme necessário.

O spaCy funciona através de pipelines, que é um fluxo de componentes ativados em sequência, onde cada pipe tem uma função específica, como identificar regras sintáticas, encontrar entidades, classificar padrões, etc. Um ponto importante levantado pelo orientador é a escolha entre usar um modelo padrão (completo, com todos os pipes) ou criar um pipeline vazio apenas com os componentes necessários. Um modelo completo oferece melhores informações, sendo mais precisas, enquanto um pipeline menor tem desempenho superior em velocidade. A

escolha correta depende do caso de uso, ou seja, quando apenas uma tarefa específica precisa ser executada e o tempo é crítico, o pipeline vazio é preferível.

Foi apresentado 3 principais componentes do spacy:

Entity Ruler: componente que permite reconhecer entidades com base em regras definidas pelo programador. Como não faz parte do pipeline padrão, precisa ser criado manualmente. É útil para pontos específicos onde o modelo geral não possui cobertura suficiente, por exemplo: nomes de cidades menores, termos médicos, nomes de personagens ou empresas de nicho.

Matcher: ferramenta para encontrar padrões específicos em um texto com base nas características linguísticas dos tokens. Diferente do NER por ir além da classificação de entidades, permitindo buscar atributos como pontuação, e-mails, sequências de classes gramaticais, etc. Uma aplicação prática demonstrada foi a identificação de falas de um personagem dentro de um texto.

Regex: a mais flexível das três ferramentas para identificar padrões baseados em sequências de caracteres. É recomendada para identificar datas, telefones, CEPs e e-mails, ou qualquer outro objeto onde possuem padrões que seguem uma estrutura previsível de caracteres. O instrutor recomenda utilizá-la em conjunto com o Entity Ruler, onde o Regex detecta o padrão e o Entity Ruler nomeia a entidade. A principal limitação do Regex é que ele opera dentro de um único token, exigindo cuidado adicional quando o padrão abrange múltiplos tokens separados.

De forma resumida:

- Regex: quando o texto segue um padrão de caracteres, como datas, números, CEPs, e-mails e telefones;
- Matcher: quando o interesse está em palavras, classes gramaticais ou sequências de tokens, aproveitando as informações linguísticas do spaCy.

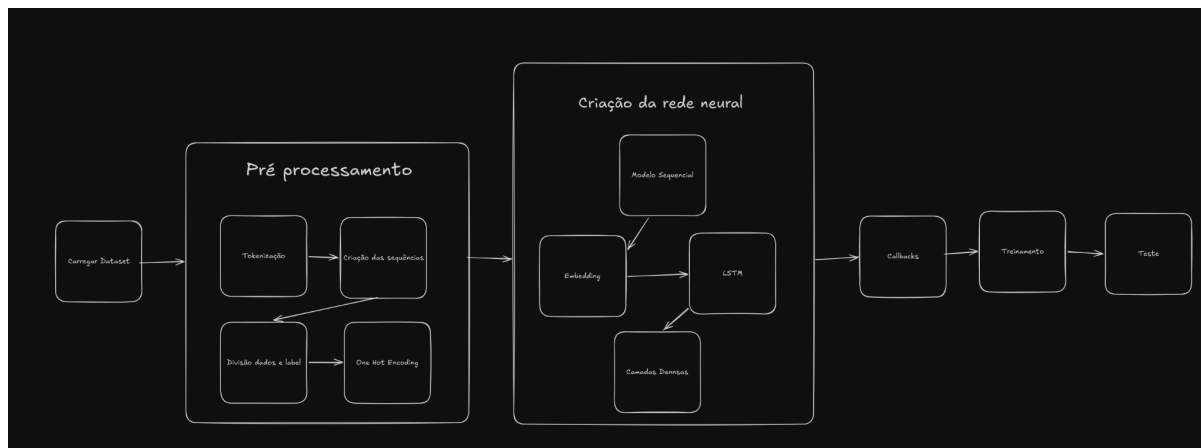
Coloquei essa diferença específica entre os dois, pois foi onde mais tive dificuldade foi quando utilizar o regex e quando utilizar o matcher.

Por fim, foi reforçado o conceito de vetores de palavras, que é exatamente o que o processo de embedding realiza: o modelo cria múltiplos vetores e transforma cada palavra em uma matriz única composta por muitos números que, juntos, conseguem carregar contextos semânticos que um único número não poderia representar.

2.3 Prática

Na prática eu fiz dois códigos, um para o spacy e um para o Tensor Flow.

O código do spacy ficou bem simples, fiz apenas testes utilizando um livro do dom casmurro e fui testando os códigos vistos em aulas para fixar o conteúdo. Quanto ao tensor flow, eu tentei criar um gerador de músicas com um dataset que encontrei com as letras de todas as músicas da Taylor Swift.



Após importar o dataset, pegamos as colunas que importam e transformamos em listas, aplicamos a tokenização, transformamos em sequências e transformamos as labels em vetores One Hot.

Para a rede neural, utilizei a camada de embedding e uma camada de LSTM, depois 4 camadas densas, onde as 2 do meio possuíam números reduzidos de neurônios.

Para os callbacks, utilizei os que gosto que são o early stopping, para parar o treinamento caso não esteja evoluindo, o reduce lr, para diminuir a taxa para continuar a evolução no treinamento, e o meu preferido que é o checkpoint, para sempre salvar o modelo em um arquivo, visto que frequentemente meu editor de código da erro e fecha, é necessário sempre salvar o modelo pra não precisar treinar de novo.

Para o teste, eu fiz uma função simples que cria uma música com base em uma frase inicial, e em um número máximo de palavras.

3. Conclusão

Esse card foi muito interessante para entender como é o processo de treinamento de uma rede neural generativa como o chat gpt ou o gemini, aprofundando ainda mais a utilização do tensorflow. Também foi ensinado como utilizar a biblioteca spacy para identificação e processamento de textos de forma rápida, decisões que devem ser tomadas e como utilizar bem essa biblioteca para extrair as informações de artigos, por exemplo.

4. Referências

Vídeos do Card